

TP: Money Laundering Analysis
75.74 Sistemas Distribuidos I
Primer cuatrimestre del 2026
Grupo 10

Nombre y Apellido	Padrón	Mail
Escandar Obadía, Germán Gonzalo	105250	gescandar@fi.uba.ar
Morseletto, Bruno Gabriel	104087	bmorseletto@fi.uba.ar
Alonso Schneider, Micaela Jazmin	109551	malonso@fi.uba.ar

Índice

Índice.....	2
Introducción.....	3
Escenarios.....	3
Vista Lógica.....	3
D.A.G.....	3
Vista de desarrollo.....	4
Diagrama de paquetes.....	4
Vista de proceso.....	5
Diagramas de actividad.....	5
Diagrama de secuencia.....	11
Vista física.....	13
Diagrama de robustez.....	13
Diagrama de despliegue.....	14
Tareas a ejecutar.....	15
División de los integrantes.....	16
Referencias.....	17

Introducción

El presente informe propone el diseño de un sistema distribuido para analizar registros de transacciones realizadas entre diferentes cuentas bancarias con el objetivo de encontrar anomalías que podrían indicar que hubo un lavado de activos.

Escenarios

El análisis de los datos consiste en poder resolver las siguientes consultas:

1. Cuenta de origen, cuenta de destino y monto para transacciones USD menores a 50.
2. Nombre de banco, cuenta de origen y monto de la máxima transacción USD de cada banco.
3. Cuenta de origen y monto de transacciones USD en el periodo [2022-09-06, 2022-09-15] con monto menor a 1 centésimo del promedio encontrado para el mismo formato de pago en el periodo [2022-09-01, 2022-09-05].
4. Cuentas que cumplan con el patrón scatter-gather con una sola cuenta de separación, para cuentas que hayan realizado y cuya cuenta de origen haya realizado transferencias USD hacia entre 5 cuentas distintas dentro del periodo [2022-09-01, 2022-09-05].
5. Cantidad de transacciones del periodo [2022-09-01, 2022-09-05] con formato de pago "Wire" o "ACH" cuyo monto convertido a USD sea menor a 1.

Vista Lógica

D.A.G.

Para observar el workflow completo, contamos con el siguiente diagrama:

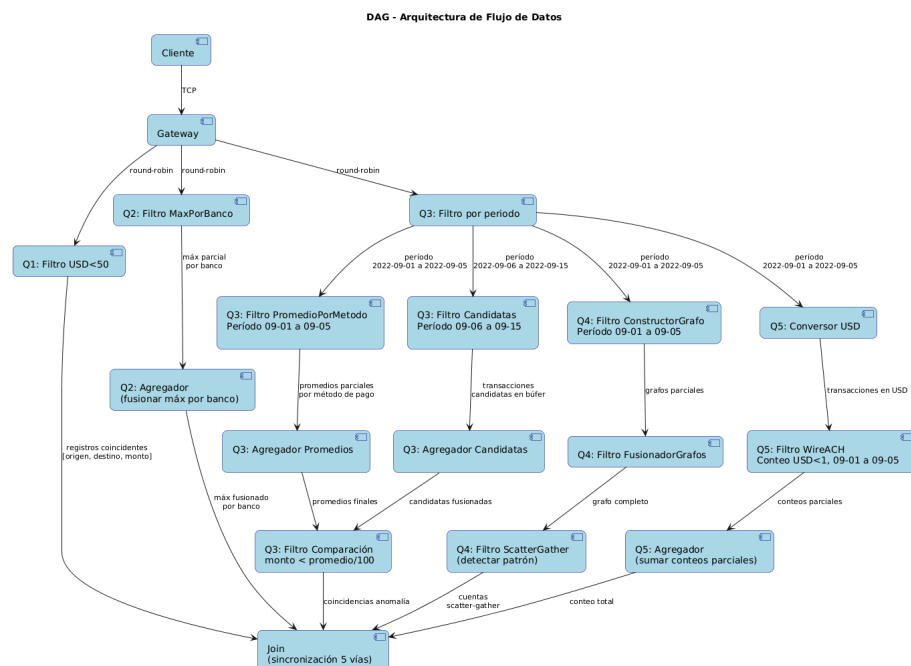


Figura 1: Diagrama D.A.G.

Se observan los componentes y filtros que actuarán como nodos en la comunicación. Cada requerimiento funcional se denota como $Q<nro\ requerimiento>$.

Vista de desarrollo

Diagrama de paquetes

A continuación se observan los diagramas correspondientes a la estructura tentativa del presente trabajo.

Por un lado, se tendrá un paquete de Common, el cual contará con el protocolo de mensajes y el Middleware del sistema. Luego, el resto del sistema será compuesto por los componentes de clientes, gateway y los diferentes workers. A modo de simplificación, se representó a los diferentes filtros, convertidores, agregadores y joins en un sólo módulo “Worker”, pues estos tendrán una similar estructura donde la única diferencia será la lógica de negocio dentro de los mismos.

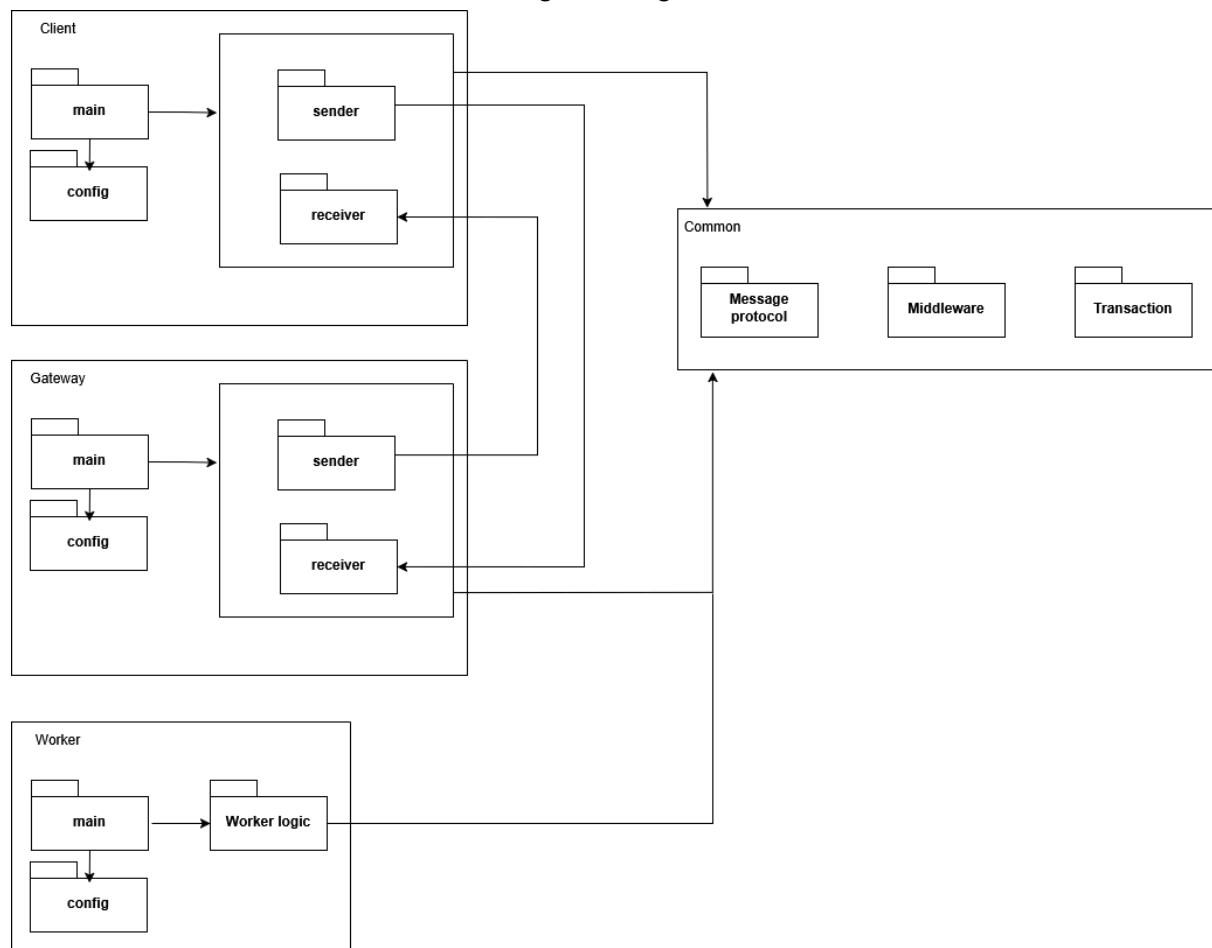


Figura 2: Diagrama de paquetes correspondiente al cliente y al gateway.

Vista de proceso

Diagramas de actividad

Los Diagramas de actividad muestran el flujo de acciones que se van tomando dentro del sistema sin tener en cuenta de manera muy detallada la implementación interna de cada componente.

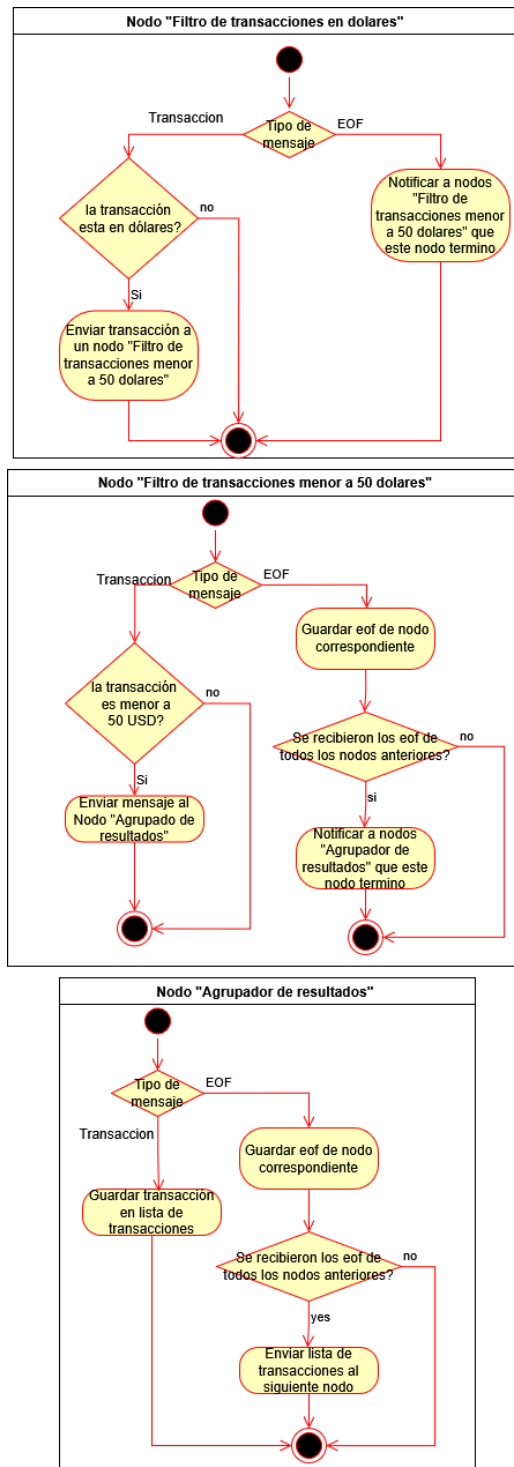


Figura 3: Diagrama de actividad de Escenario 1.

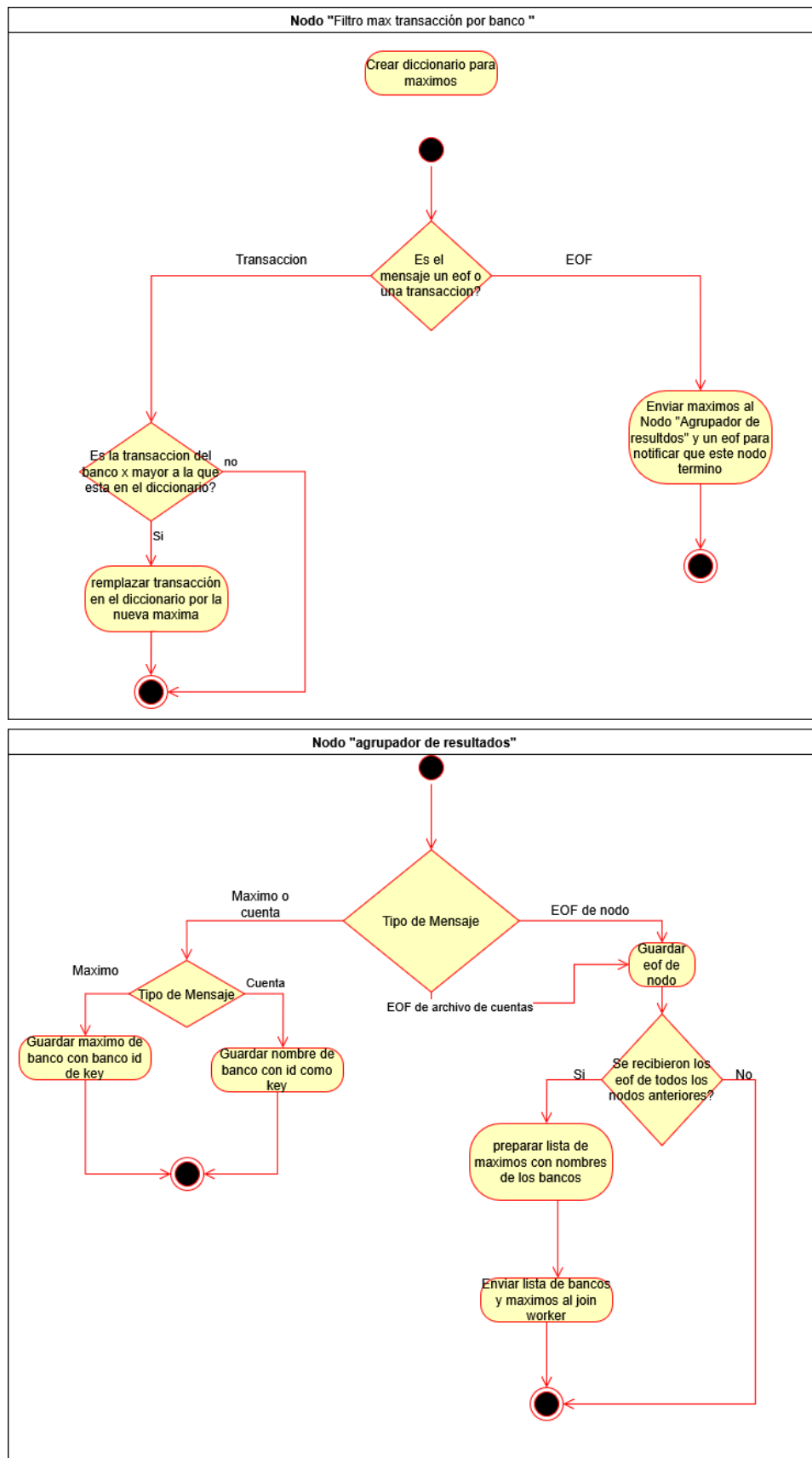


Figura 4: Diagrama de actividad del Escenario 2.

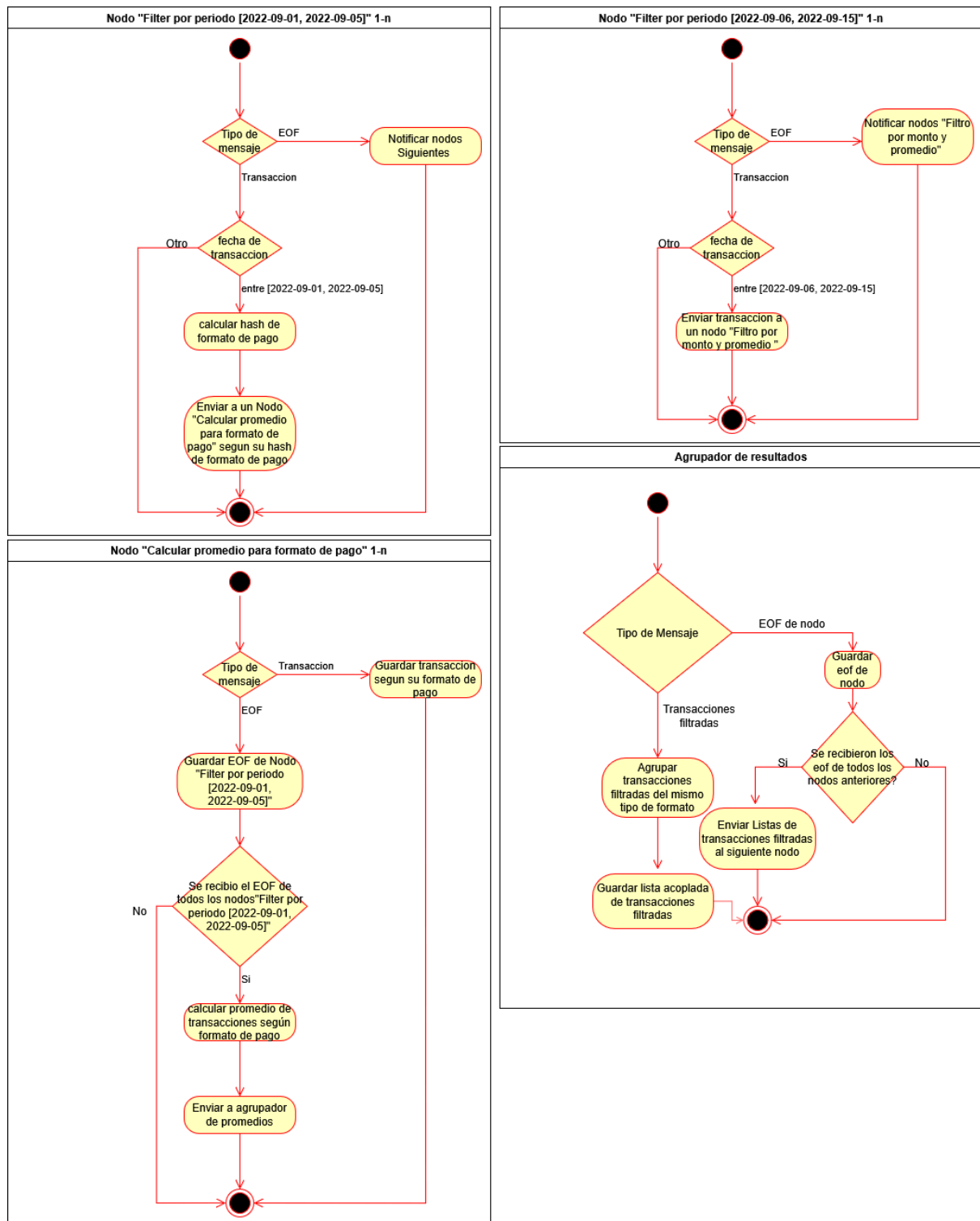


Figura 5.1: Diagrama de actividad de el Escenario 3.

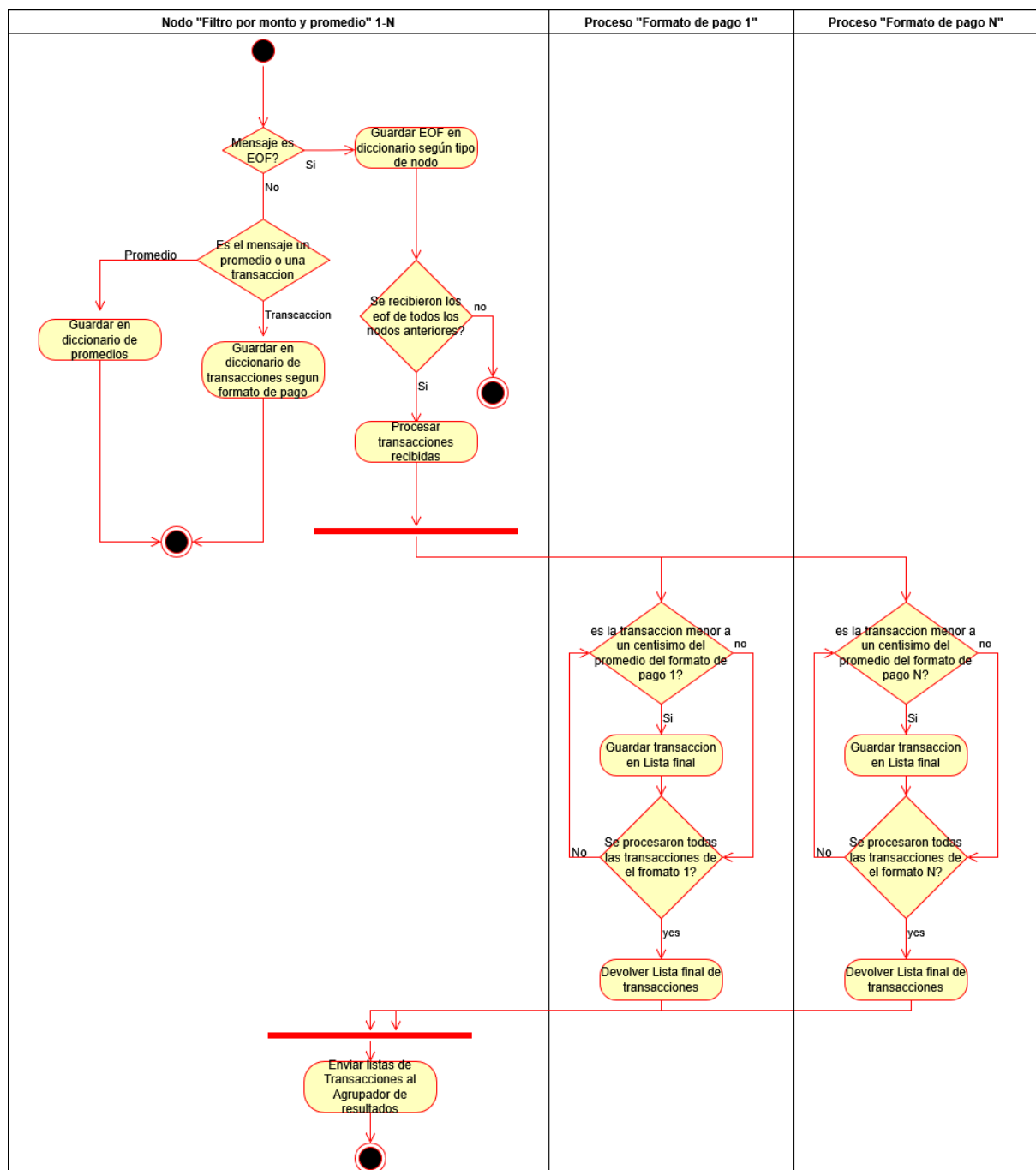


Figura 5.1: Diagrama de actividad de el Escenario 3 para el filtro por monto y promedio.

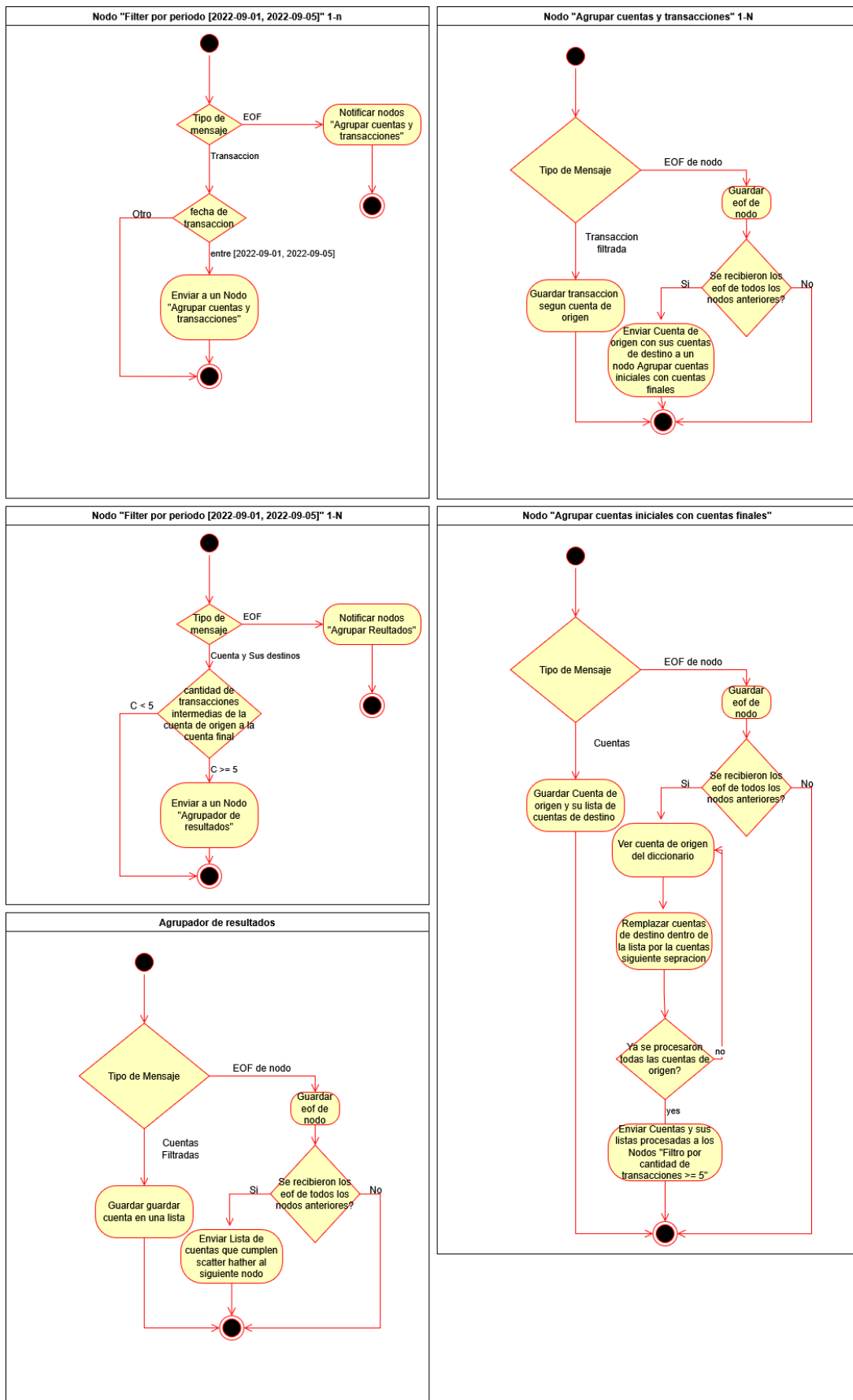


Figura 6: Diagrama de actividad del escenario 4.

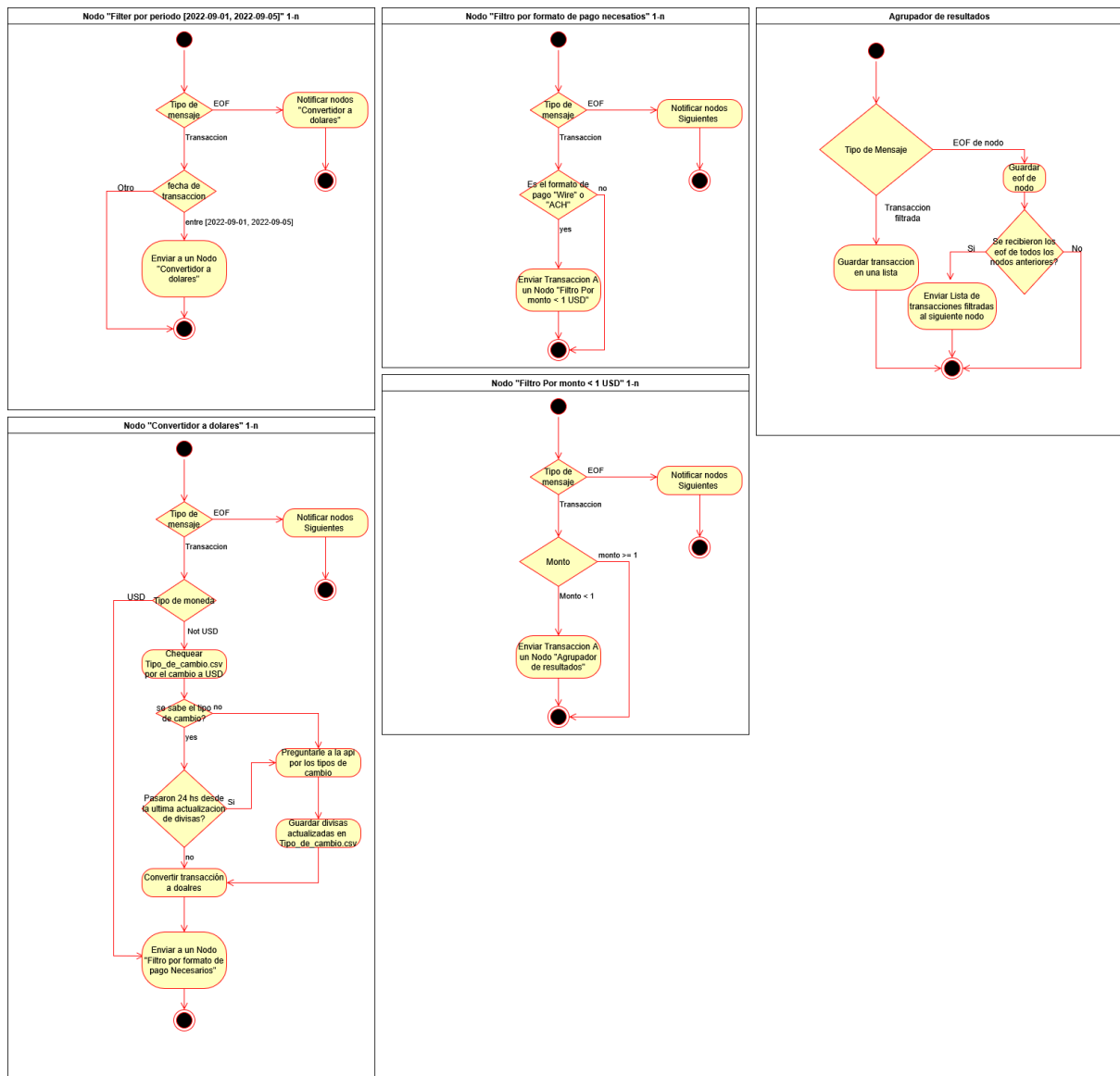


Figura 7: Diagrama de actividades del Escenario 5.

Diagrama de secuencia

Para detallar mejor el flujo mencionado anteriormente, lo dividimos en 2 partes:

- Ingesta de datos: Se lee el dataset de transacciones y se propagan los mensajes de datos hasta llegar a los filtros correspondientes.

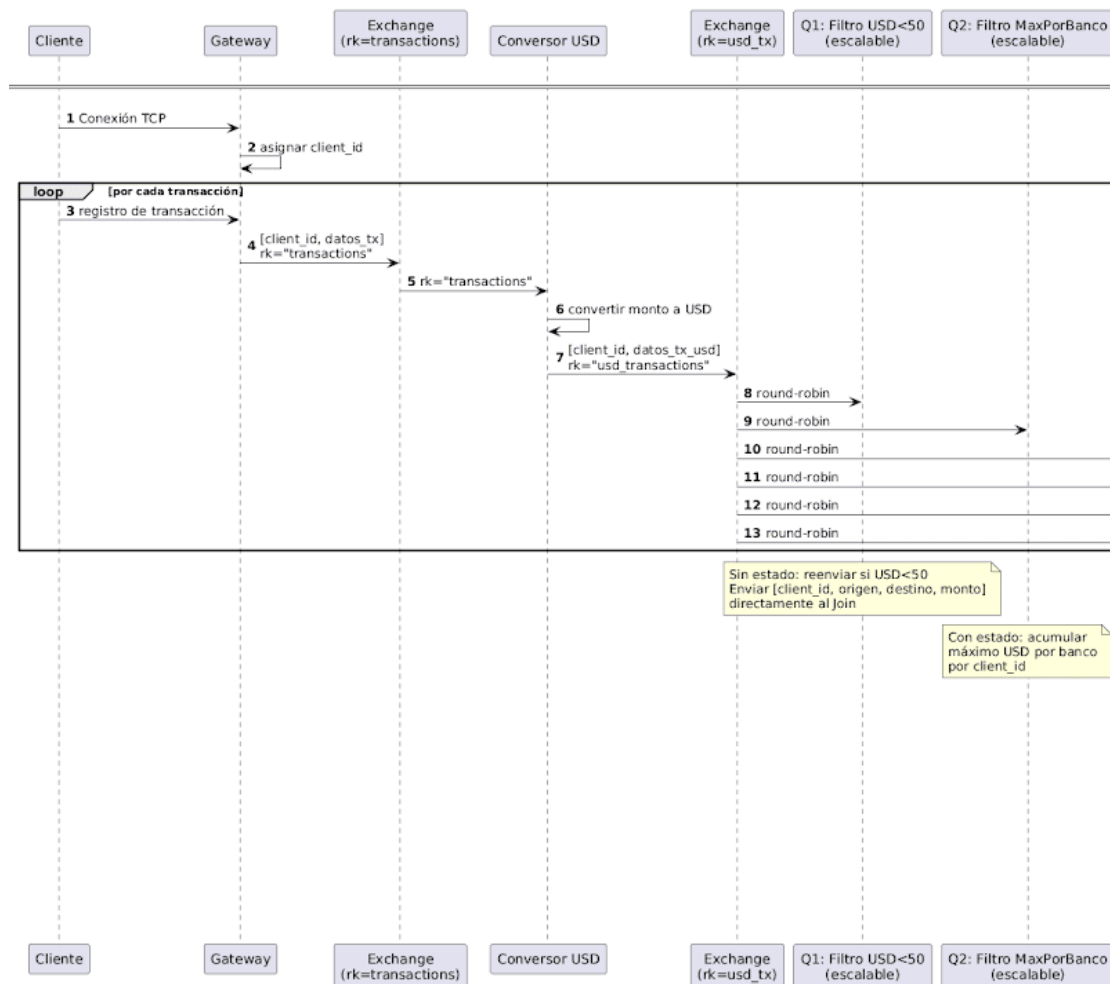


Figura 8: Primera parte del diagrama de secuencia para la ingesta de datos.

Vista física

Diagrama de robustez

En este caso se definió el posible diseño del sistema teniendo en cuenta la separación de tareas de tal manera que cada caso de uso se mantenga lo más abstraído del otro posible para reducir la propagación de errores del sistema

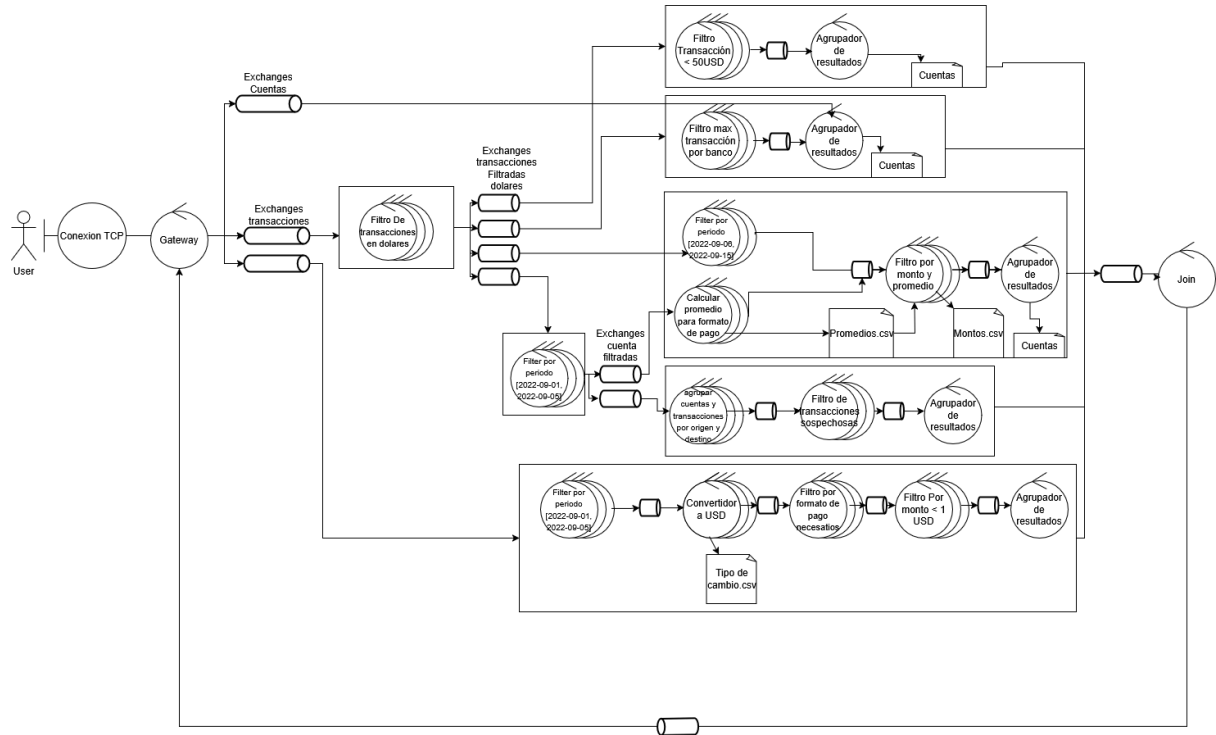


Figura 11: Diagrama de robustez del sistema.

Diagrama de despliegue

En este siguiente diagrama podemos observar las conexiones entre los componentes físicos del sistema. Dentro de las características de este sistema observamos que el cliente es un nodo aparte, el cual se comunica con el nodo Gateway. Luego, los componentes de los filtros, agregadores, convertidores y demás están conectados al componente Rabbit MQ, pues todos los nodos se comunicarán a partir de colas de mensajes.

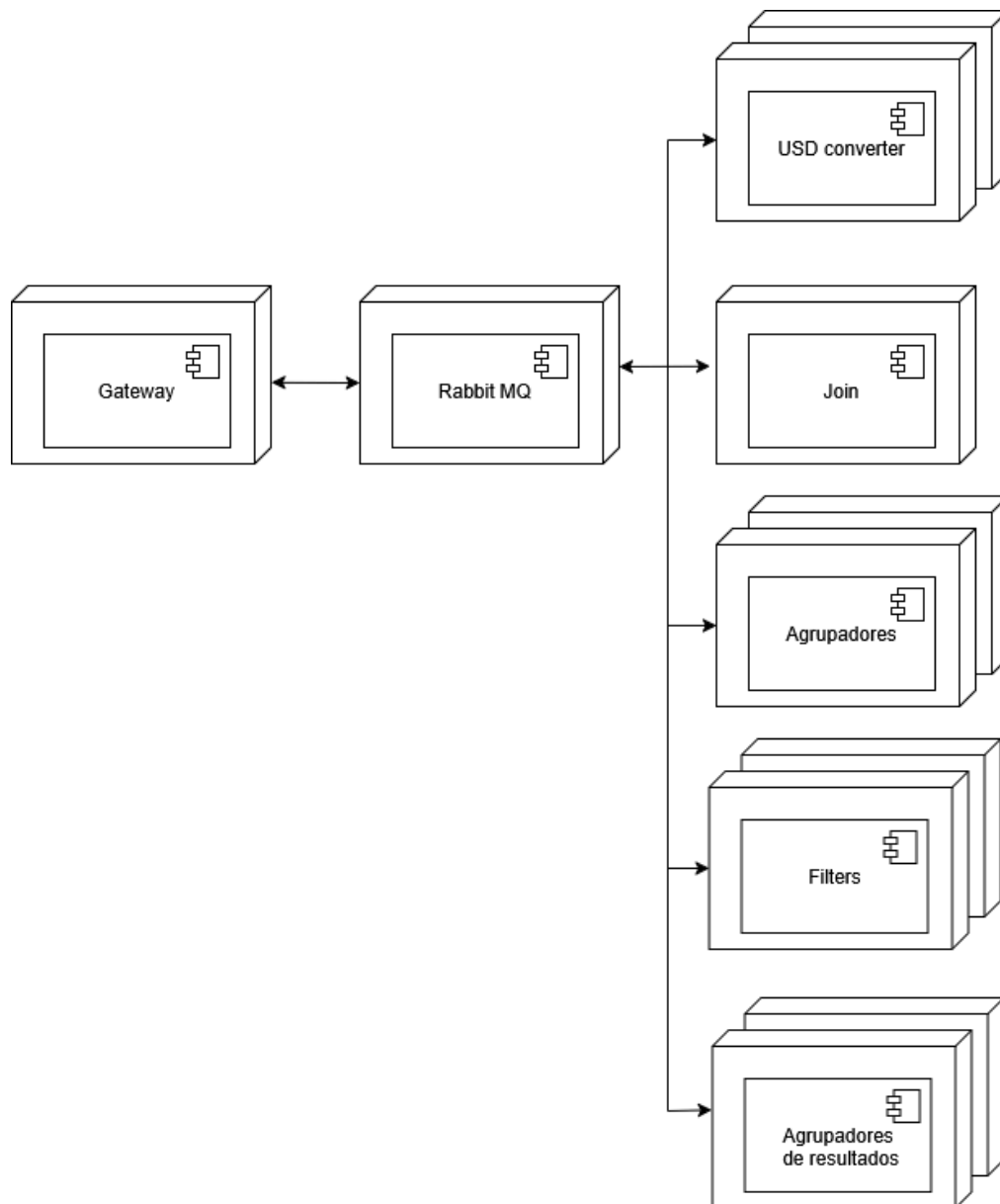


Figura 12: Diagrama de despliegue del sistema.

Tareas a ejecutar

Dentro de este trabajo se detectan las siguientes tareas:

- Middleware:
 - Configuración de la imagen de docker correspondiente al middleware.
 - Implementación middleware
- Protocolo:
 - Definición e implementación del protocolo a utilizar en los diferentes tipos de mensajes como los mensajes de envío de información y mensajes de errores.
- Cliente:
 - Configuración de la imagen de docker correspondiente a los clientes.
 - Envío de registros de transacciones hacia el gateway.
 - Recepción de los resultados.
- Gateway:
 - Implementación de interfaz con el cliente: recepción de registros y envío de resultados.
 - Recepción de resultados del Join.
 - Limpieza de los registros recibidos por el cliente: eliminación de las columnas que no son necesarias para resolver las queries.
 - Envío de los registros recibidos por el cliente hacia el Convertidor de USD.
- Convertidor de USD:
 - Lógica para consultar a la página Frankfurter para saber los valores de conversión.
 - Lógica de modificación de los registros para convertir el monto a USD.
 - Lógica de lectura y escritura de las queues.
- Filtro de monto:
 - Lógica de filtrado por monto.
 - Lógica de lectura y escritura de las queues.
- Filtro por período de tiempo:
 - Lógica de filtrado por período de tiempo.
 - Lógica de lectura y escritura de las queues.
- Filtro por formato de pago:
 - Lógica de filtrado por formato de pago.
 - Lógica de lectura y escritura de las queues.
- Calculador de promedio:
 - Lógica para calcular el promedio del monto enviado.
 - Lógica de lectura y escritura de las queues.
- Filtro de máxima transacción por banco:
 - Lógica de filtrado por máxima transacción por banco.
 - Lógica de lectura y escritura de las queues.
- Filtro por monto y promedio:
 - Lógica de filtrado por monto y promedio.
 - Lógica de lectura y escritura de las queues.
- Agrupadores:
 - Lógica de agrupación.

- Lógica de lectura y escritura de las queues.
- Agrupadores de resultados:
 - Lógica de agrupación.
 - Lógica de lectura y escritura de las queues.
- Join:
 - Lógica de agrupación para obtener los resultados finales.
 - Formateo de los resultados.
 - Lógica de lectura y escritura de las queues.

División de los integrantes

A continuación se presenta una división de tareas por cada integrante de manera preliminar:

Tarea	Integrante
Middleware	Germán Gonzalo Escandar Obadía
Protocolo	Micaela J. Alonso Schneider
Cliente	Germán Gonzalo Escandar Obadía
Gateway	Germán Gonzalo Escandar Obadía
Filtros escenarios 1-3	Bruno Gabriel Morseletto
Filtros escenarios 4-5	Micaela J. Alonso Schneider
Agrupadores	Bruno Gabriel Morseletto
Agrupadores de resultados	Bruno Gabriel Morseletto
Join	Micaela J. Alonso Schneider

Decisiones tomadas

Para el escenario número 4 se tomó las siguientes decisiones para el análisis:

- Para que el patrón scatter-gather se cumpla, la cuenta origen debe haberle enviado una transferencia a 5 o más cuentas distintas.
- El notebook provisto por la cátedra para el análisis de resultados fue modificado de la siguiente forma:
 - El merge de los datasets para obtener los pares de cuentas se realizaba haciendo un self join usando el dataset `ranged_trans_usd_sept_df` sin embargo, este dataset solo tenía las cuentas que le enviaban a 5 o más cuentas, así que se perdían las cuentas intermedias que formaban parte del patrón scatter-gather que realizaban transacciones a menos de 5 cuentas. Así que se tomó la decisión de realizar ese merge entre el dataset `ranged_trans_usd_sept_df` y el `trans_usd_sept_1st_df` (el cual tiene todas las transacciones del periodo pedido y cuyas transacciones fueron realizadas en USD).
 - Además, se eliminaron las transacciones duplicadas resultantes del merge entre los dos datasets mencionados anteriormente.

A continuación se observa el código provisto por la cátedra y el código modificado, donde lo marcado en color rojo del lado del código modificado corresponde a las partes alteradas para así lograr realizar el análisis de la interpretación del enunciado y las decisiones tomadas.

```
#4. Accounts that match the scatter-gather pattern and where the source
account has transferred to more than 5 distinct accounts.

accounts_df = ranged_trans_usd_sept_df[["From Bank", "Account", "To Bank",
"Account.1"]]
account_pairs_df = accounts_df.merge(accounts_df, left_on=["To Bank",
"Account.1"], right_on=["From Bank", "Account"]).rename(columns={
    "From Bank_x": "From Bank",
    "Account_x": "From Account",
    "To Bank_y": "To Bank",
    "Account.1_y": "To Account"
})
account_pairs_df = account_pairs_df[(account_pairs_df["From Bank"] !=
account_pairs_df["To Bank"]) | (account_pairs_df["From Account"] !=
account_pairs_df["To Account"])]
account_pairs_df = account_pairs_df.groupby(["From Bank", "From Account", "To
Bank", "To Account"], as_index=False).size()
account_pairs_df = account_pairs_df[(account_pairs_df["size"] > 5)]

from_account_pairs_df = account_pairs_df[["From Bank", "From
Account"]].rename(columns={
    "From Bank": "Bank",
    "From Account": "Account"
})
to_account_pairs_df = account_pairs_df[["To Bank", "To
Account"]].rename(columns={
    "To Bank": "Bank",
    "To Account": "Account"
})
unique_accounts = pd.concat([from_account_pairs_df,
to_account_pairs_df]).drop_duplicates()
unique_accounts
```

Código original correspondiente al escenario número 4.

```
#4. Accounts that match the scatter-gather pattern and where the source
account has transferred to more than 5 distinct accounts.

accounts_df = ranged_trans_usd_sept_df[["From Bank", "Account", "To Bank",
"Account.1"]]
account_pairs_df = accounts_df.merge(trans_usd_sept_1st_df, left_on=["To
Bank", "Account.1"], right_on=["From Bank", "Account"],
how="inner").rename(columns={
    "From Bank_x": "From Bank",
    "Account_x": "From Account",
    "To Bank_y": "To Bank",
```

```

    "Account.1_y": "To Account"
})
account_pairs_df = account_pairs_df[(account_pairs_df["From Bank"] !=
account_pairs_df["To Bank"]) | (account_pairs_df["From Account"] !=
account_pairs_df["To Account"])]
account_pairs_df = account_pairs_df.drop_duplicates().groupby(["From Bank",
"From Account", "To Bank", "To Account"], as_index=False).size()
account_pairs_df = account_pairs_df[(account_pairs_df["size"] >= 5)]

from_account_pairs_df = account_pairs_df[["From Bank", "From
Account"]].rename(columns={
    "From Bank": "Bank",
    "From Account": "Account"
})
to_account_pairs_df = account_pairs_df[["To Bank", "To
Account"]].rename(columns={
    "To Bank": "Bank",
    "To Account": "Account"
})
unique_accounts = pd.concat([from_account_pairs_df,
to_account_pairs_df]).drop_duplicates()
unique_accounts

```

Código modificado correspondiente al escenario número 4.

Luego, para el escenario número 5 se decidió tomar el valor de conversión para la moneda Bitcoin el valor provisto por la cátedra en el notebook de análisis.

Referencias

- [IBM Transactions for Anti Money Laundering \(AML\) - Kaggle](#)
- [Money Laundering Analysis - Kaggle](#)
- [Diagrama de Robustez y Diagramas de actividad](#)
- [Diagrama de Despliegue](#)
- [Diagrama de Paquetes](#)
- [Diagrama de Secuencia - Ingesta de datos](#)
- [Diagrama de Secuencia - Propagación de resultados](#)